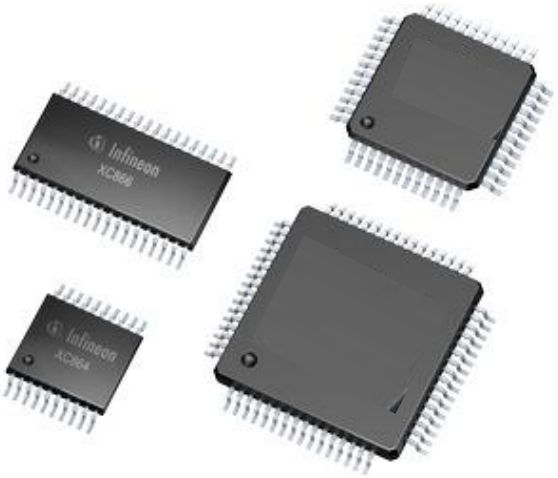
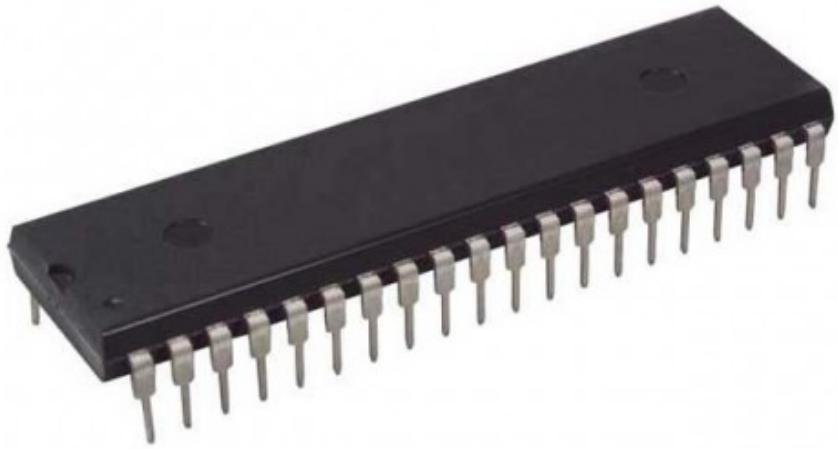
8051 Microcontroller





What is an addressing mode ?

- CPU can access data in various ways.
- Data could be in register, or in memory, or be provided as an immediate value.
- These various ways of accessing a data are called addressing mode
- Addressing mode is a way to address an operand.
- Operand : the data we are operating upon (in most cases source data)
- Various addressing modes of microcontroller are determined when it is designed and therefore can not be changed by programmer

Assembly language instruction consist of 4 fields

Label:	**mnemonic**	**operand**	**;comment**
HERE:	MOV	A,#6AH	;mov operation

Addressing modes

- Immediate
- Register
- Direct
- Register indirect
- Indexed

Immediate Addressing modes

- The source operand is a constant
- This addressing mode is named as “immediate” because it transfers an 8-bit data immediately to the accumulator (destination operand).
- The immediate data must be preceded by the pound sign, “#”
- The ‘#’ symbol before 25H indicates that operand is a data (8 bit). If ‘#’ is not present then the hexadecimal number would be taken as address.

```
MOV A, #25H   ;load 25H into A
MOV R4, #62    ;load 62 into R4
MOV B, #40H    ;load 40H into B
MOV DPTR, #4521H     ;DPTR=4512H
MOV DPL, #21H     ;This is the same
MOV DPH, #45H    ;as above
```

Register Direct Addressing Mode

MOV A, R4

- In this addressing mode we use the register name directly as source operand
- Use registers to hold the data to be manipulated

```
MOV A,R0      ;copy contents of R0 into A
MOV R2,A      ;copy contents of A into R2
ADD A,R5      ;add contents of R5 to A
ADD A,R7      ;add contents of R7 to A
MOV R6,A      ;save accumulator in R6
```

- The source and destination registers must match in size

```
MOV DPTR,A  ;will give an error
```

- The movement of data between Rn registers is not allowed

```
MOV R4,R7    ;is invalid
```

Direct Addressing Mode

•Here the address of the data (source data) is given as operand.

MOV A, 04H

•04H is the address of register 4 of register bank#0
•data stored in register 04H is moved to accumulator
•The opcode for instruction ***MOV A, address*** is E5H.

- The entire 128 bytes of RAM can be Accessed
- It is most often used the direct addressing mode to access RAM locations 30 – 7FH
- The register bank locations are accessed by the register names

```
MOV A,4       ;is same as
MOV A,R4      ;which means copy R4 into A

MOV R0,40H    ;save content of 40H in R0
MOV 56H,A     ;save content of A in 56H
```

- Only direct addressing mode is allowed for pushing or popping the stack

```
PUSH   05
POP   02
```

- Pushing the accumulator onto the stack

```
PUSH A        ;is invalid
```

must be coded as

```
PUSH   0E0H
```

Register Indirect Addressing Mode

- A register is used as a pointer to the Data
- address of the data (source data to transfer) is given in the register operand.

MOV A, @R0

- the value inside R0 is considered as an address, which holds the data to be transferred to accumulator.

- Only register R0 and R1 are used for this purpose
- R2 – R7 cannot be used to hold the address of an operand located in RAM

When R0 and R1 hold the addresses of RAM locations, they must be preceded by the “@” sign

```
MOV A,@R0    ;move contents of RAM whose address is held
             by R0 into A

MOV @R1,B    ;move contents of B into RAM whose address is
             held by R1
```

- The advantage is that it makes accessing data dynamic rather than static as in direct addressing mode
- Looping is not possible in direct addressing mode

- Since R0 and R1 are 8 bits wide, their use is limited to access any information in the internal RAM

- Whether accessing externally connected RAM or on-chip ROM, we need 16-bit pointer. In such case, the DPTR register is used

Indexed Addressing Mode

- Indexed addressing mode is widely used in accessing data elements of look-up table entries located in the program ROM
- The instruction used for this purpose is

MOVC A, @A+DPTR **MOVC A, @A+PC**

- DPTR is data pointer and PC is program counter (both are 16 bit registers)

- Use instruction MOVC, “C” means code
- The contents of A are added to the 16-bit register DPTR to form the 16-bit address of the needed data

MOVC A, @A+DPTR

The source operand is @A+DPTR
we will get the source data (to transfer) from this location
adding contents of DPTR with present content of accumulator.
addition will result a new data which is taken as the address of source data
data at this address is then transferred to accumulator.

```
ORG 00h

CLR A
MOV DPTR,#0400H
MOVC A,@A+DPTR
MOV 30H,A
INC DPTR
CLR A
MOVC A,@A+DPTR
MOV 31H,A
INC DPTR
CLR A
HERE:SJMP HERE

ORG 400H
DB 09H,08H

END
```

MOVC A, @A+PC

- works the same way as above example. The only difference is, instead of adding DPTR with accumulator, here data inside program counter (PC) is added with accumulator to obtain the target address.